Institut Supérieur d'Informatique, de Modélisation
et de leurs Applications

Génie Logiciel et Intégration d'Applications

Année 2024-2025

Travaux Pratiques

Web Serveur
Plateforme de dons d'objets

Réalisé par :
LAMINOU AMADOU Abdoul Bassit
KANE Abdou Latif

Encadré par :
Yannick Loiseau

1 Architecture du système

1.1 Principes architecturaux

L'application DonoPlatform suit une **architecture MVC** (Model-View-Controller) modulaire, organisée en couches distinctes pour assurer la séparation des préoccupations :

- **Couche Contrôleur** : Gère les requêtes HTTP entrantes (API REST et Web)
- **Couche Service** : Contient la logique métier
- **Couche Repository** : Gère l'accès aux données (JPA/Hibernate)
- **Couche Modèle** : Définit les entités JPA et les DTOs

1.2 Technologies utilisées

- **Framework** : Spring Boot 3.5.7 avec Java 17
- **Persistence** : Spring Data JPA + Hibernate
- **Sécurité** : Spring Security avec double authentification (JWT pour API, Sessions pour Web)
- **Base de données** : H2 (embedded)
- **Templates** : Thymeleaf pour le rendu serveur
- **Build** : Maven avec Maven Wrapper

1.3 Négociation de contenu

L'application implémente la **négociation de contenu** via l'en-tête HTTP `Accept` :

- `Accept: application/json` → Réponse JSON (API REST)
- `Accept: text/html` → Rendu HTML (Interface Web)

Cette approche permet le couplage faible et facilite l'intégration de clients variés (SPA, applications mobiles, CLI).

1.4 HATEOAS et hypermédia

Les réponses JSON incluent des **liens HATEOAS** (Hypermedia As The Engine Of Application State) permettant la découverte dynamique des ressources :

Listing 1 – Exemple de DTO avec liens HATEOAS

```
public class AnnouncementDto extends RepresentationModel<AnnouncementDto> {
    private Long id;
    private String title;
    // ... autres proprietes

    // Liens generes automatiquement :
    // - self : lien vers la ressource
    // - owner : lien vers le proprietaire
    // - images : lien vers les images
    // - update, delete : actions disponibles
}
```

1.5 Gestion du cache

Pour optimiser les performances et le passage à l'échelle :

- **ETags** : Génération automatique pour les ressources
- **Cache-Control** : Directives de mise en cache configurées par type de ressource
- Support des requêtes conditionnelles (`If-None-Match`)

Listing 2 – Exemple de gestion du cache

```
@GetMapping(value = "/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<ApiResponse> getAnnouncementById(
        @PathVariable Long id,
        @RequestHeader(value = "If-None-Match", required = false) String
            ifNoneMatch) {

    Announcement announcement = announcementService.getAnnouncementById(id);
```

```
    String etag = "\"" + Integer.toHexString(announcement.hashCode()) + "\"";

    if (etag.equals(ifNoneMatch)) {
        return ResponseEntity.status(HttpStatus.NOT_MODIFIED).eTag(etag).build();
    }

    return ResponseEntity.ok()
            .eTag(etag)
            .cacheControl(CacheControl.maxAge(10, TimeUnit.MINUTES).cachePublic())
            .body(new ApiResponse("Announcement found", dto));
}
```

2 Modèle de données

2.1 Entités principales

Le modèle relationnel est composé de 10 entités JPA avec leurs relations :

Entité	Description	Relations principales
User	Utilisateur de la plateforme	@OneToMany avec Announcement, SavedSearch
Announcement	Annonce de don	@ManyToOne avec User, @ManyToMany avec Keyword
Keyword	Mot-clé pour recherche	@ManyToMany avec Announcement, SavedSearch
SavedSearch	Recherche sauvegardée	@ManyToOne avec User, @OneToMany avec Notification
Favorite	Annonce favorite	@ManyToOne avec User et Announcement
Batch	Lot d'annonces groupées	@ManyToMany avec Announcement
Message	Message entre utilisateurs	@ManyToOne avec User (sender, receiver)
Notification	Notification système	@ManyToOne avec User, Announcement
Image	Image d'annonce (stockage binaire)	@ManyToOne avec Announcement
Role	Rôle utilisateur	@ManyToMany avec User

TABLE 1 – Entités du modèle de données

2.2 Relations JPA

2.2.1 Relations principales

- **User ↔ Announcement** : @OneToMany (un utilisateur peut créer plusieurs annonces)
- **Announcement ↔ Keyword** : @ManyToMany (une annonce a plusieurs mots-clés)
- **User ↔ Favorite ↔ Announcement** : relation ternaire via table de jointure
- **Batch ↔ Announcement** : @ManyToMany (regroupement d'annonces)

2.2.2 Enums

- `AnnouncementStatus` : AVAILABLE, RESERVED, GIVEN
- `ItemCondition` : NEW, LIKE_NEW, GOOD, FAIR, POOR

2.3 Schéma simplifié

Listing 3 – Exemple d'entité Announcement

```
@Entity
@Table(name = "announcements")
public class Announcement {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

```
    private String title;
    private String description;

    @Enumerated(EnumType.STRING)
    private ItemCondition itemCondition;

    @Enumerated(EnumType.STRING)
    private AnnouncementStatus status;

    private String geographicArea;
    private Boolean handDelivery;
    private Boolean canShip;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "owner_id")
    private User owner;

    @ManyToMany
    @JoinTable(name = "announcement_keywords",
               joinColumns = @JoinColumn(name = "announcement_id"),
               inverseJoinColumns = @JoinColumn(name = "keyword_id"))
    private Set<Keyword> keywords;

    @OneToMany(mappedBy = "announcement", cascade = CascadeType.ALL)
    private Set<Image> images;
}
```

3 Ressources et représentations

3.1 Schéma d'URL

Les URLs suivent une structure RESTful cohérente :

Type de ressource	Pattern d'URL
Annonces	`/announcements`, `/announcements/{id}`
Utilisateurs	`/users`, `/users/{userId}`
Recherches sauvegardées	`/search/saved`, `/search/saved/{id}`
Favoris	`/favorites`, `/favorites/user/{userId}`
Lots	`/batches`, `/batches/{batchId}`
Messages	`/messages`, `/messages/conversation`
Notifications	`/notifications`, `/notifications/unread/count`
Images	`/images/{imageId}/download`
Authentification	`/auth/login`, `/auth/register`

TABLE 2 – Schéma d'URL des ressources

3.2 Représentations par ressource

3.2.1 Annonces (`/announcements`)

Représentations gérées :

— **JSON** : AnnouncementDto avec liens HATEOAS
— **HTML** : Templates Thymeleaf (list.html, detail.html, create.html)

Méthodes HTTP :

— `GET /announcements` : Liste paginée avec tri et filtres
— `GET /announcements/{id}` : Détail d'une annonce
— `POST /announcements` : Création (JSON ou FormData)
— `PUT /announcements/{id}` : Modification complète
— `PATCH /announcements/{id}/status` : Changement de statut

— `DELETE /announcements/{id}` : Suppression

Listing 4 – Exemple de réponse JSON pour une annonce

```
{
  "message": "Announcement found",
  "data": {
    "id": 1,
    "title": "Canape en cuir",
    "description": "Canape 3 places en excellent etat",
    "itemCondition": "GOOD",
    "status": "AVAILABLE",
    "geographicArea": "Paris",
    "handDelivery": true,
    "canShip": false,
    "ownerId": 1,
    "ownerName": "John Doe",
    "keywords": ["meuble", "salon"],
    "images": [{"id": 1, "fileName": "canape.jpg"}],
    "_links": {
      "self": {"href": "http://localhost:8080/announcements/1"},
      "owner": {"href": "http://localhost:8080/users/1"},
      "images": {"href": "http://localhost:8080/images/1"},
      "update": {"href": "http://localhost:8080/announcements/1"},
      "delete": {"href": "http://localhost:8080/announcements/1"}
    }
  }
}
```

3.2.2 Recherche (`/search`)

Représentations :
— **JSON** : SearchAnnouncementRequest + Page<AnnouncementDto>
— **HTML** : Formulaire de recherche et résultats paginés
Méthodes :
— `GET /search` : Affichage du formulaire de recherche (HTML)
— `POST /search` : Exécution de la recherche avec critères
— `POST /search/saved` : Sauvegarde d'une recherche
— `GET /search/saved/user/{userId}` : Liste des recherches sauvegardées
— `PUT /search/saved/{id}` : Modification d'une recherche sauvegardée
— `DELETE /search/saved/{id}` : Suppression
Critères de recherche supportés :
— Texte libre (titre et description)
— Zone géographique
— État de l'objet
— Statut de l'annonce
— Mode de livraison (main propre / envoi)
— Mots-clés

3.2.3 Utilisateurs (`/users`)

Représentations :
— **JSON** : UserDto (sans mot de passe)
— **HTML** : Profil utilisateur, édition
Méthodes :
— `GET /users/{userId}` : Profil utilisateur
— `PUT /users/{userId}` : Modification du profil
— `DELETE /users/{userId}` : Suppression du compte

3.2.4 Authentification (`/auth`)

Méthodes :
- `POST /auth/register` : Inscription (JSON ou FormData)
- `POST /auth/login` : Connexion
  - JSON → Retourne un JWT
  - FormData → Crée une session HTTP
- `POST /auth/logout` : Déconnexion (Web uniquement)

3.2.5 Messages (`/messages`)

Représentations :
- **JSON** : MessageDto
- **HTML** : Interface de messagerie avec conversations

Méthodes :
- `GET /messages` : Liste des conversations (groupées)
- `GET /messages/conversation` : Messages d'une conversation
- `POST /messages` : Envoi d'un message
- `PATCH /messages/{id}/read` : Marquer comme lu
- `GET /messages/unread-count` : Compteur de non-lus

3.2.6 Notifications (`/notifications`)

Représentations :
- **JSON** : NotificationDto
- **HTML** : Liste de notifications avec badge de compteur

Méthodes :
- `GET /notifications` : Liste des notifications
- `GET /notifications/unread/count` : Compteur
- `PATCH /notifications/{id}/read` : Marquer comme lu
- `PATCH /notifications/read-all` : Tout marquer comme lu

Système de notification :
- Tâche planifiée exécutée toutes les 30 secondes (fixedDelay = 30000)
- Vérifie les nouvelles annonces correspondant aux recherches sauvegardées actives
- Crée automatiquement des notifications pour les utilisateurs concernés
- Badge affiché dans l'en-tête de toutes les pages HTML via intercepteur

3.2.7 Images (`/images`)

Représentations :
- **Binaire** : Image téléchargeable (image/jpeg, image/png, etc.)
- **JSON** : Métadonnées (ImageDto)

Méthodes :
- `POST /images` : Upload (multipart/form-data)
- `GET /images/{id}` : Métadonnées JSON
- `GET /images/{id}/download` : Téléchargement binaire
- `DELETE /images/{id}` : Suppression

4 Gestion de la sécurité

4.1 Double système d'authentification

L'application utilise deux chaînes de sécurité Spring Security distinctes :

4.1.1 API REST (JWT - Stateless)

— **Filtrage** : Requêtes avec `Accept: application/json`
— **Token JWT** : Passé dans l'en-tête `Authorization: Bearer <token>`
— **Durée de validité** : 1 heure (configurable)
— **Algorithme** : HMAC-SHA256
— **Claims** : id, email, roles

Listing 5 – Configuration sécurité API

```
@Bean
@Order(1)
public SecurityFilterChain apiSecurityFilterChain(HttpSecurity http) {
    http
        .securityMatcher(new RequestHeaderRequestMatcher("Accept", "
            application/json"))
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(sm -> sm.sessionCreationPolicy(
            SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/login", "/auth/register").permitAll()
            .requestMatchers("/announcements", "/search/**").permitAll()
            .anyRequest().authenticated()
        )
        .addFilterBefore(authTokenFilter,
            UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

4.1.2 Interface Web (Sessions HTTP)

— **Filtrage** : Requêtes HTML (par défaut)
— **Session** : Gestion automatique par Spring Security
— **Login** : Formulaire classique (`/auth/login`)
— **Logout** : `/auth/logout` avec invalidation de session
— **Protection CSRF** : Activée (sauf H2 console)

4.2 Rôles et permissions

— **ROLE_USER** : Utilisateur standard (création d'annonces, messagerie, favoris)
— **ROLE_ADMIN** : Administrateur (accès complet)

4.3 Hachage des mots de passe

Utilisation de **BCrypt** pour le stockage sécurisé :

Listing 6 – Encodage des mots de passe

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

5 Démarche de développement

5.1 Organisation du projet

Le projet suit une structure modulaire standard Spring Boot :

`src/main/java/fr/isima/glia/donoplatform/`

```
controller/          # Contrôleurs REST et Web
   api/             # API JSON (AnnouncementApiController, etc.)
   web/             # Contrôleurs HTML (AnnouncementWebController, etc.)
service/            # Logique métier (interfaces + implémentations)
repository/         # Accès données (JPA repositories)
model/              # Entités JPA
dto/                # Data Transfer Objects
request/            # Objets de requête
response/           # Objets de réponse (ApiResponse, JwtResponse)
security/           # Configuration sécurité et JWT
config/             # Configuration Spring (intercepteurs, etc.)
exceptions/         # Gestion centralisée des erreurs
```

5.2 Compilation et exécution

Deux scripts sont fournis pour faciliter le lancement :

5.2.1 Linux/Mac (`run.sh`)

Listing 7 – Script d'exécution Linux

```
chmod +x run.sh
./run.sh run            # Lancer l'application
./run.sh compile        # Compiler uniquement
./run.sh package        # Creer le JAR
./run.sh clean          # Nettoyer le projet
```

5.2.2 Windows (`run.bat`)

Listing 8 – Script d'exécution Windows

```
run.bat run             # Lancer l'application
run.bat compile         # Compiler uniquement
run.bat package         # Creer le JAR
```

5.3 Base de données

5.3.1 Configuration

- **Type** : H2 embedded (fichier local)
- **Localisation** : `./data/donoplatform_db`
- **Console H2** : http://localhost:8080/h2-console
- **Mode** : `create-drop` (réinitialisation à chaque démarrage)
- **Données initiales** : Créées via `DataInitializer`

Listing 9 – Configuration base de données

```
# Base de donnees H2
spring.datasource.url=jdbc:h2:file:./data/donoplatform_db
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA/Hibernate
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
```

5.4 Points d'accès

Une fois l'application lancée :

— **Interface Web** : http://localhost:8080
— **API JSON** : http://localhost:8080 (avec `Accept: application/json`)
— **Console H2** : http://localhost:8080/h2-console
  — JDBC URL : `jdbc:h2:file:./data/donoplatform_db`
  — Username : `sa`
  — Password : (vide)

5.5 Données de test

Des comptes de démonstration sont créés automatiquement au démarrage :

— **Utilisateurs** : user1@gmail.com à user5@gmail.com (mot de passe : `123456`)
— **Admins** : admin1@gmail.com, admin2@gmail.com (mot de passe : `123456`)

5.6 Validation

Validation des données via annotations Jakarta Validation :

Listing 10 – Exemple de validation

```java
public class CreateAnnouncementRequest {
    @NotBlank(message = "Le titre est obligatoire")
    @Size(max = 200)
    private String title;

    @NotBlank(message = "La description est obligatoire")
    private String description;

    @NotNull(message = "L'etat de l'objet est obligatoire")
    private ItemCondition itemCondition;

    @NotBlank(message = "La zone geographique est obligatoire")
    @Size(max = 100)
    private String geographicArea;
}
```

6 Fonctionnalités avancées

6.1 Notification automatique

Un système de notifications basé sur les recherches sauvegardées :

— **Scheduler** : Tâche planifiée (@Scheduled) exécutée toutes les 30 secondes (fixedDelay = 30000ms)
— **Logique** : Compare les nouvelles annonces avec les critères des recherches sauvegardées actives
— **Filtrage** : Exclut les annonces de l'utilisateur lui-même et celles déjà vues
— **Création** : Génère automatiquement des notifications pour les utilisateurs concernés
— **Affichage** : Badge de compteur dans l'en-tête HTML (via intercepteur Spring MVC)

Listing 11 – Scheduler de notifications

```java
@Scheduled(fixedDelay = 30000) // Toutes les 30 secondes
public void checkNewAnnouncementsForSavedSearches() {
    LocalDateTime now = LocalDateTime.now();

    List<SavedSearch> activeSearches = savedSearchRepository.findAll().
        stream()
        .filter(s -> s.getNotificationsEnabled() != null && s.
            getNotificationsEnabled())
        .toList();

    for (SavedSearch savedSearch : activeSearches) {
```

```
        SearchAnnouncementRequest request = new SearchAnnouncementRequest();
        request.setGeographicArea(savedSearch.getGeographicArea());
        request.setItemCondition(savedSearch.getItemCondition());

        Page<Announcement> results = announcementService.searchAnnouncements(
                request, PageRequest.of(0, 100));

        List<Announcement> newAnnouncements = results.getContent().stream()
                .filter(a -> a.getPublicationDate().isAfter(lastCheckTime))
                .filter(a -> !a.getOwner().getId().equals(savedSearch.getUser().getId()))
                .toList();

        for (Announcement announcement : newAnnouncements) {
            notificationService.createNotification(
                    savedSearch.getUser(), announcement, savedSearch);
        }
    }

    lastCheckTime = now;
}
```

6.2 Messagerie interne

— Conversations organisées par annonce et utilisateur
— Messages non lus marqués automatiquement
— Interface Web avec actualisation automatique (meta refresh)
— API REST pour clients mobiles

6.3 Gestion des images

— Stockage binaire en base de données (@Lob)
— Upload multipart/form-data
— Téléchargement avec en-têtes de cache appropriés
— Support de plusieurs formats (JPEG, PNG, GIF, WebP)

6.4 Intercepteur de notifications

Un intercepteur Spring MVC ajoute automatiquement le compteur de notifications à toutes les pages HTML :

Listing 12 – NotificationInterceptor

```
@Component
public class NotificationInterceptor implements HandlerInterceptor {

    @Override
    public void postHandle(HttpServletRequest request,
                           HttpServletResponse response,
                           Object handler,
                           ModelAndView modelAndView) {

        if (modelAndView != null && !isApiRequest(request)) {
            Authentication auth = SecurityContextHolder.getContext().
                getAuthentication();

            if (auth != null && auth.getPrincipal() instanceof
                DonoUserDetails userDetails) {
                Long unreadCount = notificationService.getUnreadCount(
                    userDetails.getId());
                modelAndView.addObject("notificationCount", unreadCount);
```

```
                }
            }
        }
}
```

7 Aspects techniques notables

7.1 Passage à l'échelle

— **Stateless API** : Aucune session côté serveur pour l'API REST (JWT uniquement)
— **Cache HTTP** : ETags et Cache-Control configurés pour réduire la charge
— **Pagination** : Toutes les listes utilisent Spring Data Pageable
— **Lazy Loading** : Relations JPA en FetchType.LAZY pour éviter le N+1
— **Pool de connexions** : HikariCP configuré (10 connexions max)

7.2 Extensibilité

— Architecture modulaire facilitant l'ajout de nouvelles fonctionnalités
— Interfaces de service permettant des implémentations alternatives
— Négociation de contenu permettant de nouveaux formats de réponse
— DTOs séparés des entités pour l'évolution indépendante du modèle

7.3 Sécurité

— Protection CSRF activée pour l'interface Web
— Validation des données en entrée (@Valid)
— Autorisation basée sur les rôles et propriété des ressources
— Hachage BCrypt des mots de passe
— Tokens JWT avec expiration